

雲端硬碟系統

需求與開發計畫書

整合對話式 AI 助理與時光機快照的多人雲端檔案管理系統

目 錄

一、專案概述.....	3
1.1 專案目標.....	3
1.2 預期效益.....	3
二、系統範圍與使用對象.....	3
2.1 系統範圍.....	3
2.2 使用者角色.....	4
2.3 主要使用情境.....	4
三、需求功能.....	5
3.1 功能性需求.....	5
3.2 非功能性需求.....	6
3.3 系統限制與假設.....	6
四、系統架構設計.....	7
4.1 整體架構.....	7
4.2 技術選型.....	7
4.3 系統模組與資料流程.....	8
五、功能模組詳細說明.....	8
5.1 認證與帳號模組.....	8
5.2 檔案與資料夾管理模組.....	8
5.3 上傳、下載與預覽模組.....	9
5.4 分享與權限模組.....	9
5.5 搜尋與垃圾桶模組.....	9
5.6 In-App AI 助理模組.....	9
5.7 時光機（快照）模組.....	9
六、開發環境與工具鏈.....	10
6.1 開發環境（Python uv/venv）.....	10
6.2 容器化（Docker）.....	10
6.3 版本控制（GitHub）.....	10
6.4 建議專案目錄結構.....	10
七、開發計畫與時程.....	10
7.1 開發階段劃分.....	10
7.2 關鍵節點.....	11
八、測試計畫.....	11
九、風險與對策.....	12
十、部署與維運計畫.....	12
10.1 CI/CD 架構.....	12
10.2 部署流程.....	13
10.3 維運.....	13
十一、驗收標準.....	13

一、專案概述

1.1 專案目標

本專案旨在建立一套參考 Google Drive 與 OneDrive 的多人雲端硬碟系統，讓使用者透過資料夾階層管理檔案，並能分享給其他使用者或產生公開連結。除基礎檔案管理外，本系統提供兩項進階核心能力，作為與一般雲端硬碟的差異化重點。

1. **檔案管理核心**：提供上傳、下載、預覽、資料夾、搜尋、星號、垃圾桶、版本與容量統計等完整雲端硬碟功能。
2. **對話式 AI 助理**：使用者以自然語言操作檔案，助理把需求轉成可檢視、可確認、可執行的工作流程，並能現場生成新技能。
3. **時光機（快照）**：類 Apple Time Machine 的整碟時間點還原，可瀏覽並就地還原過去某時間點的硬碟狀態。
4. **完整工程實踐**：採前後端分離、儲存層抽象、容器化部署與 CI/CD，建立可長期維護與展示的系統。

1.2 預期效益

- **操作效率**：以自然語言一句話完成批次整理、改名、移動等高頻檔案操作，降低重複手動成本。
- **資料安全**：後端集中權限檢查、metadata 與 binary 分離、敏感欄位僅存雜湊，確保使用者檔案不被越權存取。
- **可回復性**：垃圾桶、檔案版本與時光機三層保護，誤刪或誤改皆可還原至先前狀態。
- **隱私優先**：AI 助理預設以本地模型執行，資料不外流，必要時才條件式升級外部模型。
- **技術價值**：完整實踐前後端、AI agent、向量檢索、容器化與 DevOps 流程，具學習與展示價值。

二、系統範圍與使用對象

2.1 系統範圍

本系統為多人使用的雲端硬碟，使用者需登入後管理自己的檔案。系統涵蓋從「檔案上傳」到「分享、AI 操作與時光機還原」的完整流程。系統範圍內與範圍外項目如下表所示。

項目	範圍內	範圍外（本期不實作）
帳號	本系統帳號密碼登入、JWT 權杖	Google／Microsoft／學校 SSO (OAuth)
檔案操作	上傳、下載、預覽、資料夾、改名、移動、刪除	桌面同步程式、手機 App
分享協作	指定使用者分享、公開連結、權限分級	線上 Office 共同編輯、複雜企業組織權限
AI 助理	自然語言操作、計畫確認、現場生成技能	未經核可的自動執行生成程式碼
時光機	整碟快照、時間軸瀏覽、就地還原	跨使用者的全域快照管理
儲存	本機檔案系統（Storage Provider 抽象）	本期不要求接上 S3／Azure 物件儲存
安全	後端權限檢查、傳輸加密（HTTPS）	檔案內容端對端加密、防毒掃描

2.2 使用者角色

角色	說明	主要操作
一般使用者	需要管理、分享與整理個人檔案的個人。	上傳、整理、搜尋、分享、用 AI 助理操作、時光機還原。

2.3 主要使用情境

本節列舉含特殊互動或關鍵業務規則的代表性情境；完整功能清單見第三章需求功能。

情境一（上傳與整理）：使用者進入「我的硬碟」，拖曳或點選上傳檔案，後端驗證權限與容量後寫入儲存層並建立中繼資料，前端即時更新列表；可框選多個項目批次移動或刪除。

情境二（分享）：使用者選取檔案後分享給指定使用者或建立公開連結，設定可檢視／可下載／可編輯權限，並可為公開連結加上密碼與到期時間；受分享者於「與我分享」頁看到檔案。

情境三（AI 助理）：使用者以自然語言下指令（如「把上週的報告搬到 Archive 資料夾」），助理解析意圖、產生執行計畫供確認；缺少能力時現場生成技能，經核可後於沙箱執行並寫回硬碟。

情境四（時光機）：系統定期、或使用者手動、或助理執行破壞性操作前自動建立整碟快照；使用者開啟時光機瀏覽過去某時間點的硬碟，選定後就地還原整碟或特定項目，還原前自動先建保命快照。

三、需求功能

3.1 功能性需求

功能性需求描述系統必須提供的具體功能，依優先順序標記為「必要（M）／建議（S）／選用（C）」。

必要（M）— 第一版可展示與可使用的核心功能

編號	功能名稱	需求描述
FR-01	註冊與登入	使用者註冊、登入、登出；JWT 存取與刷新權杖。
FR-02	檔案上傳下載	小檔案上傳、檔案下載（串流或短效下載連結）。
FR-03	資料夾管理	建立資料夾、檔案與資料夾列表、列表／格狀檢視與排序。
FR-04	改名與移動	重新命名、移動檔案或資料夾，含同層重名衝突處理。
FR-05	垃圾桶	刪除至垃圾桶（軟刪除）、還原、永久刪除。
FR-06	搜尋	依關鍵字搜尋檔名與檔案資訊。
FR-07	星號與最近	個人化星號標記、最近開啟／修改檔案列表。
FR-08	基本預覽	圖片、PDF、文字檔等常見格式的線上預覽。
FR-09	權限檢查	後端對每個檔案操作驗證使用者權限。
FR-10	容量統計	顯示使用者容量使用狀況與上限。
FR-11	帳號設定	修改顯示名稱、登入 Email 與密碼。

建議（S）— 第二階段強化功能

編號	功能名稱	需求描述
FR-12	指定使用者分享	分享給指定使用者，權限分級：可檢視／可下載／可編輯。
FR-13	公開分享連結	建立公開連結，可選用密碼與到期時間，token 僅存雜湊。
FR-14	檔案版本紀錄	保留檔案歷史版本，支援版本回溯。
FR-15	分片上傳續傳	大檔案分片上傳、上傳進度列表與失敗續傳。
FR-16	縮圖與 PDF 預覽	圖片縮圖、PDF 預覽，縮圖於背景任務生成。
FR-17	操作紀錄	記錄上傳／下載／改名／移動／刪除等操作，供稽核與「最近」。

核心進階 (M) — 本系統差異化重點

編號	功能名稱	需求描述
FR-18	AI 助理對話操作	以自然語言列檔／搜尋／整理／改名／移動／分享／壓縮解壓。
FR-19	計畫確認	寫入與破壞性操作先產生計畫並由使用者確認，絕不自動執行。
FR-20	現場生成技能	缺少能力時現場生成技能，經靜態驗證與核可後於沙箱執行。
FR-21	技能與工作流程	側欄技能管理頁、計畫命名儲存與一鍵重跑。
FR-22	整碟快照	自動排程／手動／助理操作前自動建立整碟增量快照。
FR-23	時光機還原	時間軸瀏覽與就地還原單檔／子樹／整碟，還原前自動建保命快照。

選用 (C) — 第三階段功能

管理員後台、容量配額管理、團隊空間、共同資料夾、檔案留言與標籤、全文檢索、防毒掃描、檔案加密、OAuth 登入、WebSocket 即時通知、桌面或手機同步客戶端，列為後續可調整範圍。

3.2 非功能性需求

編號	類別	需求描述
NFR-01	效能	檔案列表分頁或虛擬滾動、搜尋輸入 debounce、下載採串流回應。
NFR-02	可用性	每個頁面皆設計 Loading／Empty／Error／權限不足／離線重試狀態。
NFR-03	可攜性	以 Docker 容器化前端、後端與資料庫，確保跨主機一致運行。
NFR-04	安全性	密碼以 Argon2 雜湊、token 與分享密碼僅存雜湊、敏感資訊以環境變數管理。
NFR-05	可維護性	前後端模組化分層、程式碼納入版本控制並具測試與文件。
NFR-06	資料保護	metadata 與 binary 分離、儲存層可抽換、後端集中權限檢查。
NFR-07	隱私	AI 助理本地模型優先、資料預設不外流，必要時才條件式升級外部模型。
NFR-08	可回復性	垃圾桶、檔案版本與時光機三層保護，破壞性操作前自動建快照。

3.3 系統限制與假設

- 系統是多人使用的雲端硬碟，使用者需登入後才能管理檔案。
- PostgreSQL 只儲存使用者、檔案中繼資料、權限、分享、版本等結構化資料，不直接儲存大型檔案二進位內容。
- 檔案本體儲存於獨立儲存層（初版用本機檔案系統，後續可換 MinIO/S3/Azure Blob）。
- 初版以網頁版為主，不含桌面同步程式與手機 App；登入採帳號密碼，OAuth 列為後續。
- AI 助理需可連線本地 Ollama (Gemma) 模型；升級外部模型時使用者須自帶加密憑證。
- 初版暫不包含線上 Office 共同編輯、端對端加密與防毒掃描。

四、系統架構設計

4.1 整體架構

系統採前後端分離：React 前端透過 HTTPS REST API 與 FastAPI 後端溝通，後端以 SQLAlchemy (async) / asyncpg 存取 PostgreSQL。檔案二進位內容不存資料庫，而是經 Storage Provider 抽象層存於獨立儲存層 (metadata 與 binary 分離)。後端另內含 AI 助理引擎，預設以本地 Gemma (Ollama) 執行，必要時升級外部模型。整體可容器化部署於 Docker 環境。

資料流為：前端 ——(HTTPS REST)——> FastAPI 後端 ——(SQLAlchemy / asyncpg)——> PostgreSQL；檔案 binary ——(Storage Provider)——> 本機檔案系統或物件儲存。取檔流程一律先查資料庫驗證權限並取得 storage_key，再向儲存層取 binary，權限與定位永遠經過資料庫，binary 不經資料庫。

4.2 技術選型

各層採用的技術與選用理由如下表（實際版本以 backend/pyproject.toml、frontend/package.json 為準）。

層級	技術	選用理由
前端	React + TypeScript + Vite	元件化 UI、型別安全；Vite 提供快速開發與建置。
前端狀態	TanStack Query + Zustand	伺服器狀態快取／失效；UI 與 auth 輕量狀態管理。
前端表單	React Hook Form + Zod	高效表單與 schema 驗證。
前端樣式	Tailwind CSS + shadcn/ui	一致設計系統、可組合元件、快速開發。
後端	FastAPI + Python 3.12	async 效能佳、原生型別、自動產生 OpenAPI。
後端 ORM	SQLAlchemy 2.x (async) + asyncpg	async ORM 搭配高效 PostgreSQL 驅動。
資料庫	PostgreSQL 16 + pgvector	關聯式資料、交易一致性與語意搜尋向量檢索。
遷移／驗證	Alembic + Pydantic	schema migration 版本管理與 I/O 驗證。
安全	PyJWT + pwdlib (Argon2) + cryptography	JWT 權杖、密碼雜湊與外部憑證加密儲存。
儲存	本機檔案系統 (Storage Provider 抽象)	開發用本地，可無痛換物件儲存。
AI 模型	Ollama (本地 Gemma) + 相容外部模型	本地優先、資料不外流；反覆失敗才升級外部。
容器化	Docker + docker compose	環境一致、一鍵啟動前後端與資料庫。
CI/CD	GitHub Actions + GHCR + 自架 Runner	自動測試／建置與手動觸發部署。

4.3 系統模組與資料流程

後端依模組（domain）組織，每個模組是自足套件並採相同分層：路由層接收請求與組裝回應、服務層集中商業邏輯（權限、容量、協調資料與儲存層）、資料層封裝 ORM 查詢與交易、結構層定義請求／回應型別。模組間只透過服務層注入互動，不互相 import 內部。

跨模組共用層包含：儲存抽象層（封裝檔案存取，可替換本地／物件儲存）、核心層（設定、JWT 安全、例外與錯誤碼、依賴注入）、資料模型層（ORM 模型與共用回應型別）、基礎服務層（操作紀錄、權限判斷、寄信等內部能力）與 API 聚合層（彙整各模組路由為單一對外 API）。

五、功能模組詳細說明

5.1 認證與帳號模組

負責註冊、登入、登出與帳號設定。採 access token 與 refresh token 雙權杖：access token 僅存於前端記憶體（不寫 localStorage/sessionStorage），refresh token 存 HttpOnly cookie；頁面重整後以 silent refresh 續期維持登入。密碼以 Argon2 雜湊，帳號設定可修改顯示名稱、登入 Email 與密碼。

5.2 檔案與資料夾管理模組

以單一 drive_items 資料表統一儲存檔案與資料夾（以 item_type 區分），支援建立、列表、改名、移動、星號與軟刪除。同層未刪除項目不可同名，同名上傳預設自動命名為 filename (1).ext。星號以 user_item_preferences 記錄，每位使用者獨立，避免分享時一人加星污染他人狀態。

5.3 上傳、下載與預覽模組

上傳分為小檔案直接上傳與大檔案分片上傳；分片上傳以狀態機（pending → uploading → completed/failed/cancelled）管理，支援續傳。上傳先寫入儲存層再建立中繼資料，資料庫流程失敗時嘗試刪除剛寫入的 blob 以避免孤兒檔案。下載採串流回應或短效下載連結並記錄操作紀錄。預覽支援圖片、PDF、文字、影片與音訊，不支援時顯示下載。

5.4 分享與權限模組

提供對指定使用者的分享（viewer/downloader/editor）與公開分享連結（可選密碼、到期與啟用開關）。分享連結只存 token 與密碼雜湊，明文 token 僅建立時回傳一次。權限判斷順序為：擁有者 → 指定分享 → 連結權限 → 資料夾繼承 → 拒絕。所有檔案操作一律於後端檢查權限，前端隱藏按鈕僅為體驗，不取代後端驗證。

5.5 搜尋與垃圾桶模組

搜尋以關鍵字過濾使用者可存取的檔名與中繼資料，名稱欄位以 pg_trgm 建索引，並可選用 pgvector 語意搜尋。刪除採軟刪除標記並移入垃圾桶，可還原（檢查父層存在與重名衝突）或永久刪除；永久刪除先移除中繼資料與配額，再由快照感知的 GC 判斷 blob 是否仍被快照引用後回收。

5.6 In-App AI 助理模組

助理後端是一個 agent harness 引擎，核心組件包含：執行迴圈、情境管理（token 預算控制與裁切）、技能與工具 registry、子代理（用於 codegen 與有界平行子任務）、系統提示組裝、生命週期 hooks（稽核、權限開、計畫確認、安裝前驗證）、持久化與模型路由。

運作流程為：使用者以自然語言下指令，助理產生可檢視的工作流程計畫；唯讀操作可 fast-path 自動執行，寫入與破壞性操作須使用者確認後才執行。缺少能力時由助理現場生成技能，經 codegen → 靜態驗證（codeguard）→ 使用者核可 → 受限沙箱執行，產出檔案寫回硬碟。已安裝技能依 manifest 動態掛到右鍵選單，常用計畫可命名儲存並一鍵重跑。模型預設本地 Gemma（Ollama），達失敗上限且符合隱私條件時才條件式升級外部模型。

5.7 時光機（快照）模組

提供類 Apple Time Machine 的整碟時間點還原。快照記錄某時間點硬碟的完整狀態（檔案／資料夾、名稱、位置、版本），並以 checksum_sha256 共用未變更內容做增量儲存。觸發方式有三種：自動排程（預設每小時）、手動立即建立、以及助理執行寫入或生成式 skill 前自動建立。

使用者可於時間軸瀏覽任一快照（唯讀），並就地還原單檔、資料夾子樹或整碟，子樹／整碟還原可選保留現有新新增（keep_new）或精確鏡像（exact_mirror）。還原前自動先建「還原前保命快照」可再倒回。快照空間不計入檔案配額，另設獨立快照配額（預設為檔案配額的一半），保留最近 N 個快照（預設 50，釘選與保命快照豁免）。

六、開發環境與工具鏈

6.1 開發環境 (Python uv/venv)

後端以 uv 搭配 pyproject.toml 管理 Python 3.12 依賴與虛擬環境，並以 ruff (lint/format)、mypy (型別檢查)、pytest (測試) 維持品質。前端以 npm 管理依賴，使用 Vitest、Testing Library、MSW 與 Playwright 進行測試。

6.2 容器化 (Docker)

前端與後端各自撰寫 Dockerfile，並以 docker compose 統一編排前端 (nginx)、後端 (FastAPI) 與資料庫 (pgvector/pgvector:pg16) 三項服務。檔案儲存以 volume 掛載，本機開發可一鍵啟動完整環境。

6.3 版本控制 (GitHub)

專案自始即以 Git 進行版本控制並推送至 GitHub。透過 .gitignore 排除虛擬環境、相依套件、環境變數檔、本機儲存與快取，確保倉庫整潔與敏感資訊安全。

- 應排除項目：.venv/、node_modules/、.env、本機 storage、PostgreSQL volume、__pycache__/、coverage 產物。
- 敏感資訊：JWT secret、資料庫密碼、LLM API 金鑰與憑證加密金鑰僅存 .env 或 secret manager，並提供 .env.example 作為範本。

6.4 建議專案目錄結構

採前後端分離的單一倉庫：backend/ (FastAPI 應用、依模組組織的 app/、Alembic migrations、tests/、eval/)、frontend/ (React 應用、src/ 依頁面與元件組織、e2e/)、docker-compose.yml、.github/workflows/ (CI 與 CD) 與 .env.example。

七、開發計畫與時程

7.1 開發階段劃分

專案採漸進式開發，以四週推進：先完成專案基礎與帳號，再做檔案核心，接著分享與預覽，最後強化與驗收；AI 助理與時光機兩項核心進階能力在檔案核心穩定後接續展開。各階段重點如下表。

階段	重點工作	產出
W1 基礎與帳號	前後端專案、Docker Compose、PostgreSQL；FastAPI 與 React 基礎；註冊登入與 JWT；drive_items 資料表與 migration。	可登入的系統骨架
W2 檔案核心	資料夾與列表 API、小檔上傳下載、容量檢查、改名移動、星號、最近、垃圾桶、搜尋、右鍵選單。	可用的我的硬碟
W3 分享與預覽	指定使用者分享、公開連結、與我分享頁；圖片／PDF／文字預覽。	分享與預覽功能
W4 強化與驗收	分片上傳與續傳、權限與效能優化、錯誤處理、測試補齊、部署文件與 Demo。	可驗收的系統
進階 AI 助理	harness 引擎、計畫確認、技能生成與沙箱、技能管理頁、工作流程重用。	對話式 AI 助理
進階 時光機	快照資料表與排程、時間軸瀏覽、就地還原與保命快照、配額。	整碟快照還原

7.2 關鍵節點

5. 節點一（第 1 週末）：完成專案基礎，使用者可註冊、登入並建立資料夾。
6. 節點二（第 2 週末）：檔案核心打通，可於瀏覽器完整完成上傳、整理、搜尋與垃圾桶流程。
7. 節點三（第 3 週末）：完成分享與預覽，兩個帳號可跨頁完成分享流程。
8. 節點四（第 4 週末）：通過驗收測試並完成容器化部署；AI 助理與時光機兩項核心進階能力完成並通過評測。

八、測試計畫

測試分為前端單元、後端單元與整合、端對端（E2E）三層，並對 AI 助理另設獨立評測 harness。

層級	工具	測試重點
前端單元	Vitest + Testing Library + MSW	登入表單驗證、檔案列表渲染、上傳進度、右鍵選單、分享彈窗、搜尋 debounce、錯誤訊息、AI 助理面板。
後端單元／整合	pytest (PostgreSQL 整合)	註冊登入、JWT、建立資料夾、上傳下載、權限拒絕、分享、搜尋、垃圾桶還原、容量限制、分片上傳。
E2E	Playwright	登入、建立資料夾、上傳、搜尋、分享、另一帳號開啟分享、刪除與還原。
AI 助理評測	backend/eval (YAML 案例)	工作流程／狀態／安全斷言、多次執行通過率與變異、baseline 回歸；in-process mock 進 CI、可選真實模型與 Browser runner。
回歸防護	前後端不變式測試	token 不得寫入 localStorage、Router HTTP 狀態碼正確、上傳自動建立版本 v1、核心元件 props 介面。

九、風險與對策

針對開發過程中可能遭遇的風險預先識別並擬定對策，如下表所示。

風險	影響	因應對策
大檔案上傳失敗	使用體驗差	分片上傳與失敗續傳。
權限判斷錯誤	資料外洩	後端集中權限檢查與完整測試。
檔案名稱衝突	使用者困惑	明確的同層重名衝突策略與提示。
資料夾樹查詢過慢	列表載入慢	建立索引，必要時引入 ltree 或 closure table。
AI 助理誤執行破壞性操作	資料損毀	計畫確認、操作前自動建快照、生成程式碼絕不自動執行。
生成程式碼安全風險	系統被濫用	codeguard 靜態掃描 + 使用者核可 + 受限沙箱。
分享連結外流	資料風險	密碼、到期時間與撤銷機制，token 僅存雜湊。
儲存成本上升	維運成本高	容量限制、垃圾桶清理與快照配額。
AI 隱私外洩	資料外送風險	本地模型優先、隱私開控管，必要時才條件式升級外部。

十、部署與維運計畫

10.1 CI/CD 架構

採 GitHub Actions 搭配自架 Runner。CI 由 GitHub 託管 Runner 執行：每次 PR/push 跑後端 pytest/ruff/mypy 與前端 lint/test/build 及 Docker build 檢查，通過並合併 main 後建置正式 image，以 Git commit SHA 標記推送至 GHCR；正式 image 只由 CI 產生。CD 由部署主機上的自架 Runner 以手動觸發（workflow_dispatch）領取，登入 GHCR、執行固定部署腳本（pull → up -d → 健康檢查 → 失敗回滾）。

10.2 部署流程

9. feature branch 推送並開 PR，CI 通過且經審查後合併 main。
10. 合併後 CI 重跑測試、建置前後端 image，以 commit SHA 推送 GHCR。
11. 於 GitHub Actions 手動觸發 Deploy，輸入要部署的 commit SHA。
12. 自架 Runner 領取工作、拉取該 SHA image、啟動服務並執行健康檢查，成功或自動回滾。

10.3 維運

- 健康檢查：部署後輪詢後端健康端點，連續失敗即判定部署失敗並自動回到上一個可用 image SHA。
- 正式設定：.env 與正式 compose 設定只存於部署主機、不進 GitHub；密鑰由 secret manager 或受控環境注入。
- 對外暴露：僅前端 nginx 為公開入口並終止 TLS，後端、資料庫與 LLM 僅限內網存取。
- 備份與監控：定期備份 PostgreSQL 資料與快照、監控 API 用量與容器健康狀態，以 docker compose 管理服務生命週期。

十一、驗收標準

功能以第三章必要（M）需求為基準，除功能完整外需同時滿足以下品質門檻。

安全與隔離：使用者只能存取自己的檔案，不能藉猜測 UUID 存取他人資源；未授權操作回傳正確錯誤碼（401/403）且不洩漏資源是否存在；分享連結 token 不可預測且可設失效。

品質與體驗：前端清楚呈現 Loading/Empty/Error 三種狀態；容量超限、同名衝突等邊界情況有明確提示。

測試與部署：第八章規劃的前端單元/E2E 與後端單元/整合測試通過；AI 助理評測 harness 通過；Docker 開發環境可一鍵啟動。